

Playwright UI Automation Test Report

Project Name: __UI Automation Testing with Playwright

Participant: Shahbaz Ahmad

Date: 18-07-2026

Branch: shahbaz-final-capstone

Assessment: Final Capstone

Question NO 2

Objective

The objective of this automation project is to validate the core functionality of the tripStack application using Playwright with website

<https://tripstack.doomple.com/>

The automated test suite verifies critical user workflows — login, flight search, and search validation — and ensures that the application behaves as expected across supported browsers. The framework is designed to provide reliable, maintainable, and scalable UI automation.

Github Link

My Branch name is **shahbaz-final-capstone** and link is [ust-sdet-training/SDET at shahbaz-final-capstone](#)

User Story

As a user, I want to log in and search for flights successfully so that I can complete my booking without errors and receive the expected results from the system.

As a user, I want to perform the application workflow successfully so that I can complete my task without errors and receive the expected results from the system.

1. Home page of is visible .
2. Search text box is working as per given parameter
3. Negative search that is not matched
4. Empty search
5. Filter flight
6. Select seat
7. Fill details
8. Book ticket
9. Check the PNR

Environment Setup

- Tools: Node.js, Playwright Test Runner, TypeScript/JavaScript, VS Code
- Install Commands:
- npm init playwright@latest
 - npm install dotenv
 - npm install winston
 - npx playwright install
 - Reporting Tools: HTML Reporter, Allure

Framework Architecture

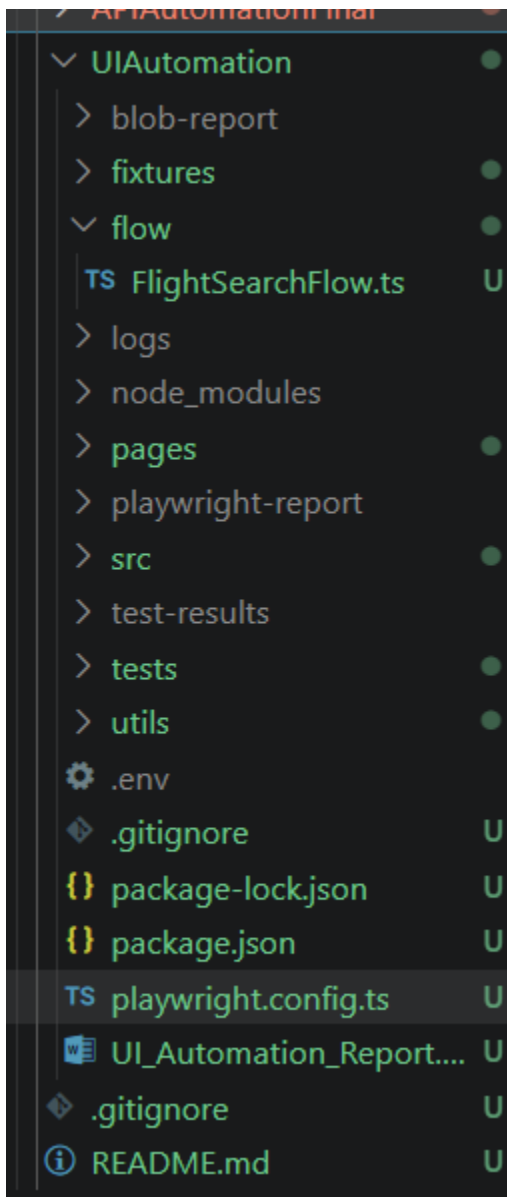
The framework is built using:

- Page Object Model (for UI abstraction)(Homepage)
- Fixtures (for reusable setup like login, credentials)(tests)
- Test files (business scenarios)(flight.spec.ts)
- Config file (playwright.config.ts)

Workflow

1. Start test execution
2. Load fixtures (page,test,expect)
3. Navigate using page.goto(<https://tripstack.doomple.com/>)
4. Use Page Object methods
5. Execute assertions
6. Generate report

Folder Structure



Page Object Model

Each UI page is represented as a class containing locators and methods. This improves maintainability and avoids duplication. I create HomePage for visible home page and search negative and positive actions.

BasePage

Respective Page

Flow

Test(with log evidence and screenshot)

Fixtures Usage

Fixtures are used to provide reusable resources like home page, session. So that can access home page method and we can use POM Homepage in test easily assert all the things by using home

```

private filterPage: FlightFilterPage;
private resultsPage: FlightResultsPage;
private seatPage: SeatSelectionPage;
private loginPage: LoginPage;

constructor(private page: Page, private log: AppLogger) {
  this.homePage = new HomePage(page, log);
  this.filterPage = new FlightFilterPage(page, log);
  this.resultsPage = new FlightResultsPage(page, log);
  this.seatPage = new SeatSelectionPage(page, log);
  this.loginPage = new LoginPage(page, log);
}

async loginUser(email: string, password: string) {
  await this.loginPage.login(email, password);
}

async searchAndBook(baseUrl: string) {
  this.log.info("===== Flight Search Flow Started =====");

  await this.homePage.open(baseUrl);
  await this.homePage.verifyHomePage();
  await this.homePage.selectFrom("Bengauluru", "BLR");
  await this.homePage.selectTo("Kolkata", "CCU");
  await this.homePage.selectDate("2026-07-29");
  await this.homePage.searchFlights();

  await this.filterPage.filterByAirlines(["IndiGo", "Vistara", "SpiceJet", "Air India"]);
  await this.filterPage.filterByTimeSlots([
    "Before 6 AM",
    "AM 6 12 PM",
    "PM 6 PM",
    "After 6 PM",
  ]);
}

```

```

async searchAndBook(baseUrl: string) {
    this.log.info("===== Flight Search Flow Started =====");

    await this.homePage.open(baseUrl);
    await this.homePage.verifyHomePage();
    await this.homePage.selectFrom("Bengauluru", "BLR");
    await this.homePage.selectTo("Kolkata", "CCU");
    await this.homePage.selectDate("2026-07-29");
    await this.homePage.searchFlights();

    await this.filterPage.filterByAirlines(["IndiGo", "Vistara", "SpiceJet", "Air India"]);
    await this.filterPage.filterByTimeSlots([
        "Before 6 AM",
        "AM 12 PM",
        "PM 6 PM",
        "After 6 PM"
    ]);
    await this.filterPage.setMaxPrice("641306");

    await this.resultsPage.bookFlight("IndiGo 6E-494");

    await this.seatPage.selectSeat("Seat 3A, window, available");

    this.log.info("===== Flight Search Flow Completed =====");
}
}

```

Test Case ID	Description	Status
--------------	-------------	--------

TC_001	Verify Home Page	Passed
--------	------------------	--------

TC_002	Verify Valid Flight Search	Passed
--------	----------------------------	--------

Negative Scenarios

Test Case ID	Description	Status
TC_003	Verify Invalid Product Search	Passed
TC_004	Verify Empty Search Validation	Passed

Commands

- `npx playwright test --headed`
- `npx playwright show-report`

Result Evidence

Attach screenshots, videos, and trace files generated during execution.

In cmd run the command and it run all 4 test pass

```
✓ 1 [chromium] › tests\bus.spec.ts:5:5 › Search Bus (8.5s)

1 passed (13.4s)

To open last HTML report run:

npx playwright show-report
```


How to Solve Issues

Check locators specific for element(search,home logo,shirt)

Wait for url,load content

Auto retries and use auto wait

fixture setup,

test data, and

network conditions.

Limitations

Dependent on UI stability, network speed, and environment configuration and number of user so ,we only run here in headed mode so that url can easily fetch

Conclusion

Playwright framework ensures scalable, maintainable, and fast UI automation using modern testing practices.

Today I did home page verification ,search text working with positive search like product list is available and negative search like not found and empty search;